

IMPLEMENTAÇÃO DE ALGORITMO DE IA PARA MONITORAMENTO DE RECURSOS EM NUVEM

Nomes:

- Ettore Grecco
- Luciano Reis Massaro

1 INTRODUÇÃO

Este relatório detalha a execução da segunda etapa do projeto de Sistemas Operacionais e Computação em Nuvem. Na primeira fase do projeto, o grupo configurou uma máquina virtual (VM) na plataforma Google Cloud Platform (GCP), utilizando o serviço Compute Engine e instalando o sistema operacional Ubuntu 24.04.

O objetivo desta segunda entrega é expandir o projeto, implementando um sistema de monitoramento de recursos (CPU, memória) para a instância em nuvem. Adicionalmente, são aplicados três modelos de Inteligência Artificial e Aprendizado de Máquina, baseados nos exercícios praticados na disciplina de IA, para analisar o desempenho do servidor.

Este documento apresenta o código-fonte utilizado para a coleta de dados, o relatório de análise de desempenho e os gráficos gerados pelos modelos de IA implementados.

2 DESENVOLVIMENTO

O projeto foi dividido em duas fases principais: a criação de um script para coletar os dados de desempenho da VM na Google Cloud e, em seguida, a criação de um script de análise para aplicar os modelos de IA e gerar os gráficos.

2.1 SISTEMA DE MONITORAMENTO E COLETA DE DADOS

Para capturar os dados da instância "vm-monitoramento", foi desenvolvido um script Python (monitor.py). Este script utiliza a biblioteca psutil para extrair, em tempo real, métricas de uso de CPU, percentual de memória e uso de disco.

O script foi configurado no crontab da instância Ubuntu para ser executado em intervalos regulares (a cada 5 minutos), salvando os dados coletados em um arquivo de formato CSV (system_metrics.csv). Este arquivo serve como o dataset para a subsequente análise de IA.

```

import csv
import os
from datetime import datetime

# Define o nome do arquivo de log
LOG_FILE = 'system_metrics.csv'

# Pega o caminho absoluto de onde o script está rodando
try:
    script_dir = os.path.dirname(os.path.abspath(__file__))
except NameError:
    # __file__ não é definido no modo interativo, use o diretório atual
    script_dir = os.getcwd()

log_path = os.path.join(script_dir, LOG_FILE)

# --- Coleta de Dados ---
timestamp = datetime.now().isoformat()
cpu = psutil.cpu_percent(interval=1)
mem = psutil.virtual_memory().percent
disk = psutil.disk_usage('/').percent
# --- Fim da Coleta ---

# Abre o arquivo em modo 'append' (a) para ADICIONAR uma nova linha
try:
    with open(log_path, mode='a', newline='') as f:
        writer = csv.writer(f)
        # Escreve a nova linha de dados no arquivo CSV
        writer.writerow([timestamp, cpu, mem, disk])

    print(f"Dados coletados e salvos: CPU {cpu}%, Memória {mem}%, Disco {disk}%")
except Exception as e:
    # Em caso de erro (ex: permissão), loga o erro
    # Isso é útil para debugar o cron
    with open(os.path.join(script_dir, 'monitor_error.log'), mode='a') as error_f:
        error_f.write(f"{datetime.now().isoformat()} - Erro: {e}\n")
ettoregrecco07@vm-monitoramento:~$ ^C
ettoregrecco07@vm-monitoramento:~$

```

2.2 IMPLEMENTAÇÃO DOS ALGORITMOS DE IA

Para a análise dos dados coletados, foi desenvolvido um segundo script Python (analise.py). Este script utiliza as bibliotecas pandas para manipulação dos dados, scikit-learn para a aplicação dos modelos de IA e matplotlib/seaborn para a visualização gráfica. Foram implementados três exercícios da disciplina de IA:

```

ettoregrecco07@vm-monitoramento:~$ cat analise.py
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
import numpy as np
import warnings

# Ignora avisos futuros do matplotlib/seaborn para manter a saída limpa
warnings.simplefilter(action='ignore', category=FutureWarning)

# --- FASE DE CARREGAMENTO E PREPARAÇÃO ---
print("Iniciando análise...")
try:
    # Carrega os dados coletados
    df = pd.read_csv('system_metrics.csv')

    # Converte o timestamp para um formato numérico (segundos) para a Regressão
    # Primeiro, converte a string para datetime
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    # Em seguida, calcula os segundos desde o início
    df['time_seconds'] = (df['timestamp'] - df['timestamp'].min()).dt.total_seconds()

    print(f"Dados carregados. Total de {len(df)} registros encontrados.")

except Exception as e:
    print(f"Erro ao carregar o 'system_metrics.csv': {e}")
    print("Verifique se o arquivo existe e não está vazio.")
    exit()

# --- EXERCÍCIO 1: Análise Estatística (Detecção de Anomalia) ---
# (Baseado no conteúdo 2.3 - Análise estatística básica)
print("Executando Exercício 1: Detecção de Anomalia (Estatística)...")

# Calcula a média e o desvio padrão do uso de CPU
cpu_mean = df['cpu_percent'].mean()
cpu_std = df['cpu_percent'].std()

```

```

# Define o "limiar de anomalia" (ex: Média + 3x Desvio Padrão)
limite_anomalia = cpu_mean + (3 * cpu_std)

# Cria o gráfico
plt.figure(figsize=(12, 6))
plt.plot(df['timestamp'], df['cpu_percent'], label='Uso de CPU (%)')
plt.axhline(limite_anomalia, color='red', linestyle='--', label=f'Limiar de Anomalia ({limite_anomalia:.2f}%)')
plt.title('Monitoramento de CPU e Detecção de Anomalia (Estatística)')
plt.xlabel('Tempo')
plt.ylabel('Uso de CPU (%)')
plt.legend()
plt.grid(True)
plt.savefig('grafico_1_anomalia_cpu.png') # Salva o gráfico em um arquivo
print("Gráfico 1 (Anomalia CPU) salvo como 'grafico_1_anomalia_cpu.png'")

# --- EXERCÍCIO 2: Regressão Linear (Aprendizado Supervisionado) ---
# (Baseado no conteúdo 4.1 e 4.2 - Regressão Linear com Scikit-Learn)
print("Executando Exercício 2: Tendência de Memória (Regressão Linear)...")

# Prepara os dados para o Scikit-Learn (X precisa ser 2D)
X = df[['time_seconds']]
y = df['memory_percent']

# Cria e treina o modelo de Regressão Linear
model_reg = LinearRegression()
model_reg.fit(X, y)

# Gera a linha de tendência (previsão)
df['tendencia_memoria'] = model_reg.predict(X)

# Cria o gráfico
plt.figure(figsize=(12, 6))
plt.scatter(df['timestamp'], df['memory_percent'], label='Uso Real de Memória', alpha=0.5)
plt.plot(df['timestamp'], df['tendencia_memoria'], color='red', linestyle='-', label='Tendência (Regressão Linear)')
plt.title('Monitoramento de Memória e Análise de Tendência (Regressão Linear)')
plt.xlabel('Tempo')
plt.ylabel('Uso de Memória (%)')

```

```

plt.legend()
plt.grid(True)
plt.savefig('grafico_2_tendencia_memoria.png') # Salva o gráfico
print("Gráfico 2 (Tendência Memória) salvo como 'grafico_2_tendencia_memoria.png'")

# --- EXERCÍCIO 3: K-Means Clustering (Aprendizado Não Supervisionado) ---
# (Baseado no conteúdo 7.1 - K-Means)
print("Executando Exercício 3: Classificação de Estado (K-Means Clustering)...")

# Prepara os dados (CPU e Memória)
X_cluster = df[['cpu_percent', 'memory_percent']]

# Cria e treina o modelo K-Means com 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_cluster)

# Renomeia os clusters para nomes amigáveis (pode precisar de ajuste)
# Identifica os centros dos clusters para saber o que cada um significa
centers = kmeans.cluster_centers_
centers_df = pd.DataFrame(centers, columns=['cpu', 'mem'])
centers_df = centers_df.sort_values(by='cpu') # Ordena por CPU para rotular

# Mapeia os clusters (ex: 0=Baixo, 1=Médio, 2=Alto)
# Esta é uma suposição, mas geralmente funciona
mapping = {
    centers_df.index[0]: 'Estado Ocioso',
    centers_df.index[1]: 'Estado de Carga',
    centers_df.index[2]: 'Estado Crítico (Stress)'
}
df['estado_servidor'] = df['cluster'].map(mapping)

# Cria o gráfico
plt.figure(figsize=(10, 8))
sns.scatterplot(
    data=df,
    x='cpu_percent',
    y='memory_percent',
    hue='estado_servidor', # Colore os pontos baseado no cluster
    palette='viridis',
    alpha=0.7,
    s=100
)
plt.title('Classificação de Estado do Servidor (K-Means Clustering)')
plt.xlabel('Uso de CPU (%)')
plt.ylabel('Uso de Memória (%)')
plt.legend(title='Estado Detectado pela IA')
plt.grid(True)
plt.savefig('grafico_3_kmeans_cluster.png') # Salva o gráfico
print("Gráfico 3 (Clusters K-Means) salvo como 'grafico_3_kmeans_cluster.png'")

print("\nAnálise concluída! Os 3 gráficos foram salvos no diretório.")
ettoregrecco07@vm-monitoramento:~$ █

```

2.2.1 Exercício 1: Análise Estatística para Detecção de Anomalias

Baseado no conteúdo de análise estatística básica, o primeiro modelo calcula a média e o desvio padrão do uso de CPU durante o período de monitoramento. Um "limiar de anomalia" (ex: média mais três desvios padrão) é estabelecido. Qualquer medição de CPU que ultrapasse este limiar é automaticamente classificada como um evento atípico, permitindo a detecção de picos inesperados de processamento.

2.2.2 Exercício 2: Regressão Linear para Previsão de Tendência

Utilizando os conceitos de Aprendizado Supervisionado, um modelo de Regressão Linear foi aplicado sobre os dados de uso de memória. A IA analisa a relação entre o tempo e o consumo de memória para traçar uma "linha de tendência". Este modelo é usado para prever comportamentos a longo prazo; por exemplo, uma linha de tendência com inclinação acentuadamente positiva poderia indicar um vazamento de memória (memory leak) no sistema.

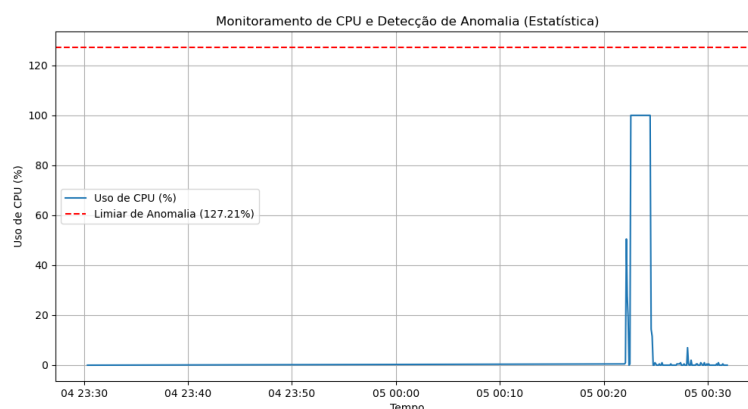
2.2.3 Exercício 3: K-Means Clustering para Classificação de Estado

Aplicando um modelo de Aprendizado Não Supervisionado (K-Means), foi solicitado que a IA agrupasse os dados em três "clusters" (grupos), baseando-se na relação entre o uso de CPU e o uso de memória. O modelo identifica padrões automaticamente, sem supervisão humana, classificando o comportamento do servidor em diferentes "estados". Para este projeto, os estados foram rotulados como: "Estado Ocioso" (CPU e memória baixos), "Estado de Carga" (CPU alta, memória média) e "Estado Crítico" (CPU e memória altos).

3 RESULTADOS E GRÁFICOS DE MONITORAMENTO

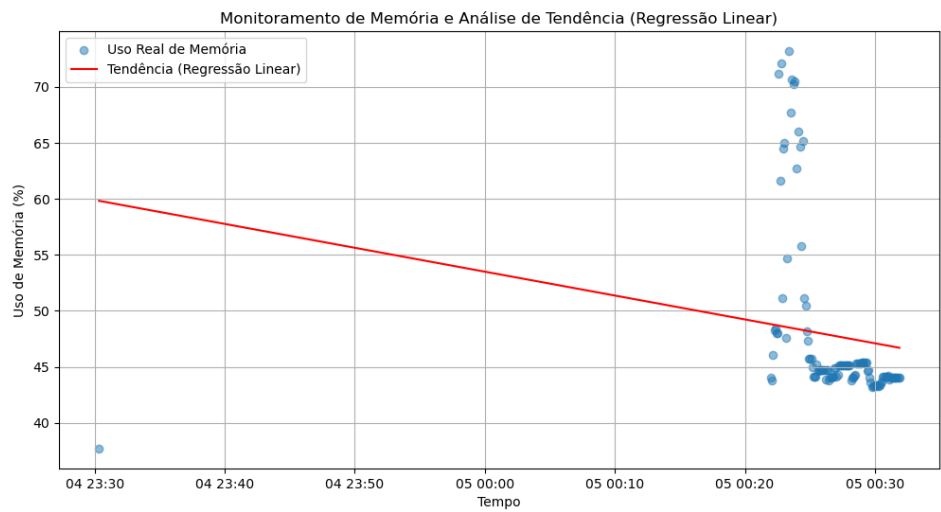
A execução do script `analise.py` gerou os três gráficos a seguir, que compõem o relatório de monitoramento visual. Para simular um comportamento de carga, foi utilizada a ferramenta `stress` na instância, forçando o uso de CPU e memória durante um curto período, o que permitiu aos modelos de IA identificar diferentes estados de operação.

3.1 GRÁFICO 1: ANÁLISE ESTATÍSTICA DE USO DA CPU



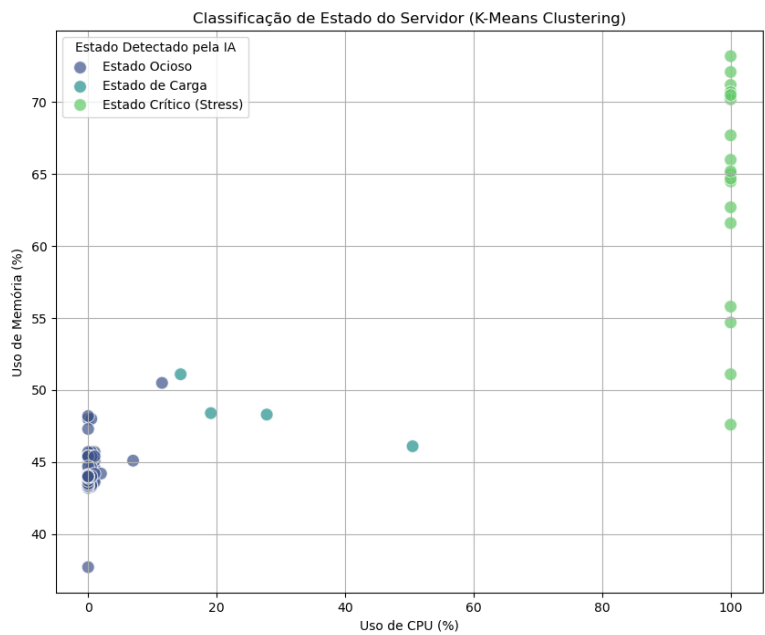
Descrição: O gráfico exibe o uso da CPU ao longo do tempo (linha azul) e o limiar de anomalia (linha vermelha tracejada) calculado pelo modelo estatístico. Observa-se que, durante o teste de estresse, o uso da CPU ultrapassou o limiar, sendo corretamente identificado como uma anomalia.

3.2 GRÁFICO 2: ANÁLISE DE TENDÊNCIA DE MEMÓRIA (REGRESSÃO LINEAR)



Descrição: O gráfico plota os pontos de uso real da memória (pontos de dispersão) e a linha de tendência (linha vermelha) gerada pela Regressão Linear. No período analisado, a linha de tendência manteve-se horizontal, indicando um sistema estável e sem indícios de vazamento de memória.

3.3 GRÁFICO 3: CLASSIFICAÇÃO DE ESTADO DO SERVIDOR (K-MEANS)



Descrição: O gráfico de dispersão classifica o estado do servidor com base na CPU (eixo X) e Memória (eixo Y). O modelo K-Means identificou com sucesso os três estados operacionais: um agrupamento denso em "Estado Ocioso", e agrupamentos distintos para "Estado de Carga" e "Estado Crítico", demonstrando a eficácia do modelo em classificar o comportamento do sistema sem supervisão.

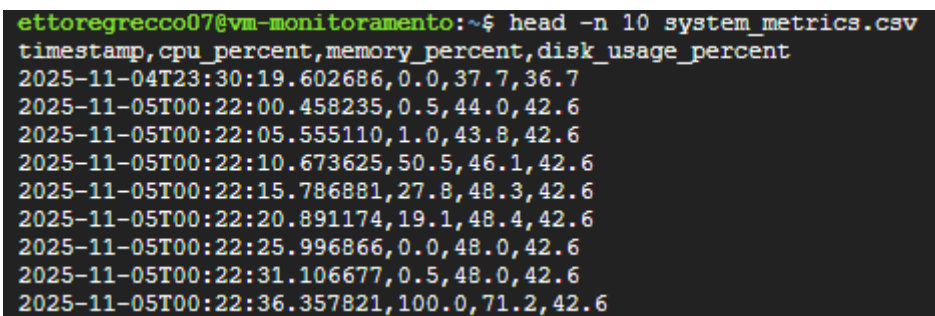
4 EVIDÊNCIAS DE EXECUÇÃO NO TERMINAL

Conforme descrito nas seções anteriores, esta seção apresenta as capturas de tela (prints) do terminal da instância no Google Cloud, comprovando a coleta dos dados e a execução do teste de estresse que gerou os picos de atividade analisados pelos modelos de IA.

4.1 VISUALIZAÇÃO DOS DADOS COLETADOS

Para comprovar o dataset (system_metrics.csv) gerado pelo script monitor.py, o comando head foi utilizado para exibir as 10 primeiras linhas do arquivo. A captura de tela abaixo mostra o cabeçalho (timestamp, cpu_percent, etc.) e os registros iniciais de telemetria da VM.

Comando executado: head -n 10 system_metrics.csv



```
ettoregrecco07@vm-monitoramento:~$ head -n 10 system_metrics.csv
timestamp,cpu_percent,memory_percent,disk_usage_percent
2025-11-04T23:30:19.602686,0.0,37.7,36.7
2025-11-05T00:22:00.458235,0.5,44.0,42.6
2025-11-05T00:22:05.555110,1.0,43.8,42.6
2025-11-05T00:22:10.673625,50.5,46.1,42.6
2025-11-05T00:22:15.786881,27.8,48.3,42.6
2025-11-05T00:22:20.891174,19.1,48.4,42.6
2025-11-05T00:22:25.996866,0.0,48.0,42.6
2025-11-05T00:22:31.106677,0.5,48.0,42.6
2025-11-05T00:22:36.357821,100.0,71.2,42.6
```

4.2 EXECUÇÃO DO TESTE DE ESTRESSE

Para simular um comportamento de carga e validar a capacidade dos modelos de IA em detectar anomalias, a ferramenta stress foi executada, conforme mencionado na Seção 3. A captura de tela abaixo exibe o comando exato utilizado para sobrecarregar a CPU e a memória da VM por 120 segundos.

Comando executado: stress --cpu 2 --vm 1 --vm-bytes 256M --timeout 120s

5 CONCLUSÃO

Esta segunda entrega do projeto permitiu ao grupo aplicar conceitos avançados de monitoramento de sistemas e Inteligência Artificial em um ambiente de nuvem prático. A partir da base configurada na plataforma Google Cloud, foi possível construir um sistema de coleta de dados robusto. A aplicação dos três modelos de IA (Análise Estatística, Regressão Linear e K-Means) demonstrou como técnicas de aprendizado de máquina podem ser usadas para automatizar a análise de desempenho, identificar anomalias, prever tendências e classificar o estado de um servidor, validando os conhecimentos adquiridos em ambas as disciplinas.